

# The SVD of the Fredy dependency matrix

9135 declarations · 126 847 dependency edges · computed 2026-07-03

## The matrix

$A$  is  $9135 \times 9135$  with  $A_{ij} = 1$  when declaration  $i$  **uses** declaration  $j$  (the constants `#print` axioms walks), else 0. It is very sparse: 126 847 ones out of 83 million entries (0.15%). Read each **row** as one declaration’s checklist of what it uses.

The eigenvalues are useless here: the dependency graph has no cycles, so  $A$  is (after reordering) triangular with a zero diagonal and **every eigenvalue is 0**. The SVD works anyway — singular values are always real and  $\geq 0$ .

## What the SVD is

The SVD writes the whole matrix as a sum of simple blocks, biggest first:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T + \dots$$

Each  $u_r v_r^T$  is an outer product — a rank-1 matrix whose  $(i, j)$  entry is one number,  $\sigma_r \cdot u_r[i] \cdot v_r[j]$ : the block’s guess for “**does declaration  $i$  use dependency  $j$** ”, a per-declaration number times a per-dependency number. So one block is **a group of declarations that share a group of dependencies**, and  $\sigma_r$  says how large that block is. The three pieces:

- $v_r$  (a column of  $V$ ): a weighting over the 9135 **dependencies** — a bundle of things used together.
- $u_r$  (a column of  $U$ ): a weighting over the 9135 **declarations** — who draws on that bundle.
- $\sigma_r$  (a diagonal entry of  $S$ , zero off-diagonal): the size of block  $r$ .

The vectors are unit length, so  $u_r$ ’s entries are tiny (spread over 9000 declarations) while  $v_r$ ’s concentrate on a few dependencies. Signs are arbitrary (flip  $u_r$  and  $v_r$  together);  $v_2, v_3$  come out as **contrasts** (some +, some −), which is how a block separates two groups.

## The singular values, and why $\sigma_1^2$ is 14%

| $r$                       | 1            | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   | 11   | 12   |
|---------------------------|--------------|------|------|------|------|------|------|------|------|------|------|------|
| $\sigma_r$                | 133.2        | 81.8 | 57.7 | 53.3 | 50.9 | 48.3 | 45.4 | 43.2 | 39.5 | 38.9 | 36.3 | 35.3 |
| $\sigma_r^2 / \Sigma$ (%) | <b>13.98</b> | 5.28 | 2.63 | 2.24 | 2.04 | 1.84 | 1.63 | 1.47 | 1.23 | 1.19 | 1.04 | 0.98 |

The “% of what” is the point. The squared size of the matrix is

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = (\text{number of ones}) = 126847,$$

because the entries are 0 or 1, so  $A_{ij}^2 = A_{ij}$ . A basic SVD identity says this same number is the sum of the squared singular values:

$$\|A\|_F^2 = \sum_r \sigma_r^2 = 126847.$$

So each  $\sigma_r^2$  is a slice of one fixed total. The first block:

$$\sigma_1^2 = 133.2^2 = 17742, \quad 17742/126847 = 13.98\% \approx 14\%.$$

That is what “14%” means: **14% of all the 1s in the matrix are accounted for by the single biggest co-usage block**. No other block is close (next is 5.3%), and it takes all top 12 to reach 35.5% — the structure is spread thin, not concentrated in a few blocks.

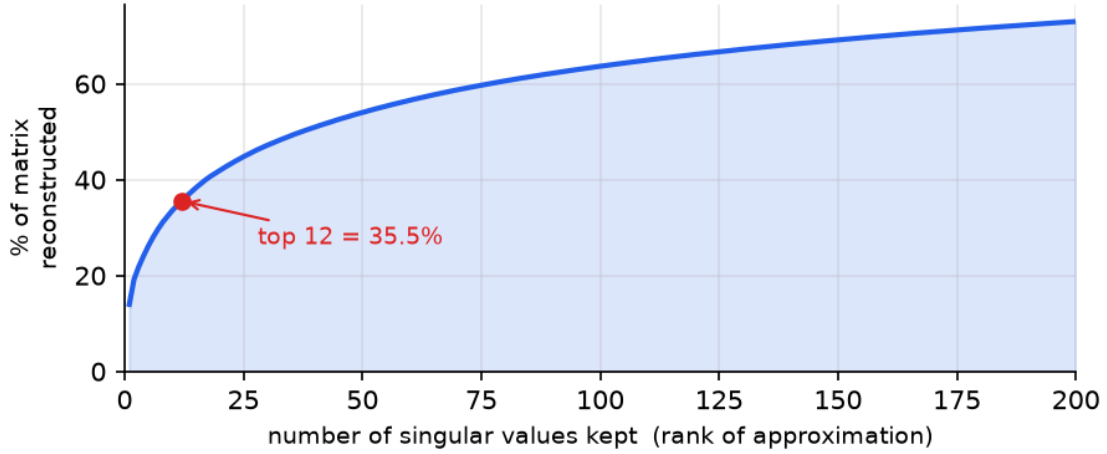


Figure 1: Adding up  $\sigma_r^2/\Sigma$ : how much of the matrix the top- $r$  blocks reconstruct. Top 12 reach only 35.5%; you need 100 blocks for 64%, 200 for 73%. The climb is slow because the matrix is high-rank.

### Can we rebuild $A$ from the top 12? No.

Keeping the top  $r$  blocks gives the **best rank- $r$  approximation** of  $A$  ( $A_r = \sum_{s=1}^r \sigma_s u_s v_s^T$ ) — but that is still a full  $9135 \times 9135$  matrix: it uses **all 9135 entries** of each  $u_s$  and  $v_s$ , not a  $12 \times 12$  corner. And “best rank-12” only gets 35.5% of the way (the plot). So no — the top 12 do not reproduce most of  $A$ ; the dependency matrix is genuinely high-rank (many small, roughly equal blocks), so you would need hundreds of  $u, v$  pairs.

**Does the top-left  $12 \times 12$  of  $A$  make sense to show?** Not as it stands: the declarations are ordered by file then line, so the top-left corner is just the 12 alphabetically-first files’ lemmas — unrelated to the SVD, and not the rank-12 approximation either. What **is** meaningful is the submatrix on 12 **chosen** important declarations. Below: the 12 heaviest core-users (largest entries of  $u_1$ ) against the 12 core dependencies (largest entries of  $v_1$ ), read off  $A$  directly (1 = uses it):

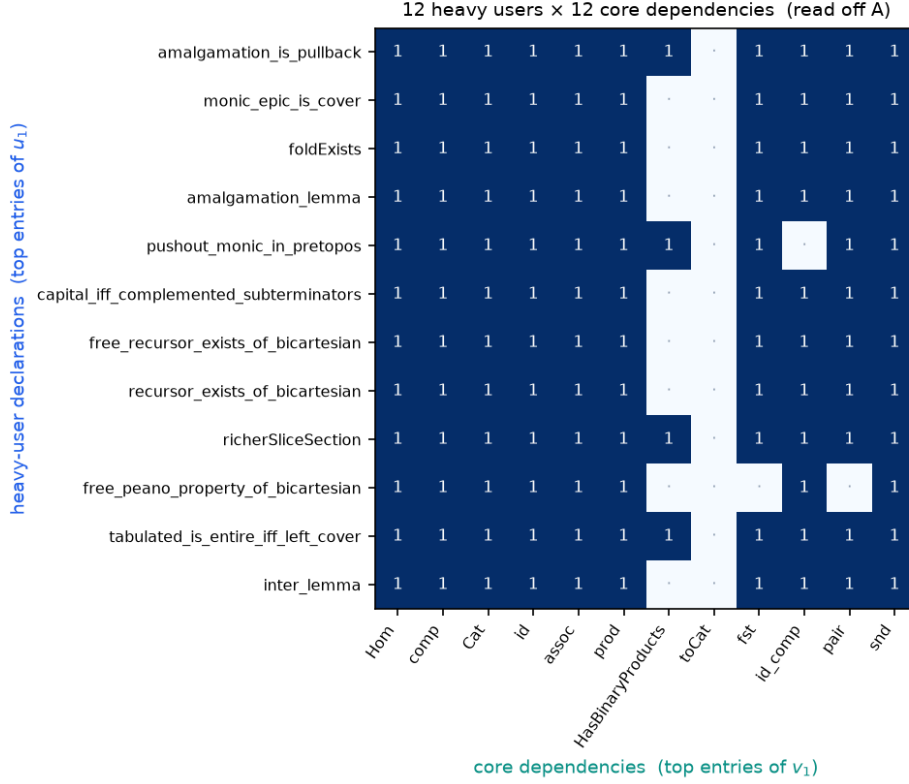


Figure 2: The near-solid block that  $\sigma_1 u_1 v_1^T$  approximates: 84% ones — the heavy users use almost the whole category core. The one empty column is `Allegory.toCat`: these are category (not allegory) theorems, so they skip the allegory bridge — real structure, visible directly in  $A$ .

So to see the SVD you don't look at a  $12 \times 12$  of  $A$ ; you look at how the top  $u$  and  $v$  pick out a dense block like this one. Shrinking  $A$  to  $12 \times 12$  is impossible — every  $u_r, v_r$  spans all 9135 declarations.

## The top three dependency bundles $v_1, v_2, v_3$

$v_1$  —  $\sigma_1 = 133.2$  (14%)  
the category core

+0.51 `Cat.Hom`  
+0.41 `Cat.comp`  
+0.39 `Cat`  
+0.21 `Cat.id`  
+0.20 `Cat.assoc`  
+0.14 `prod`  
+0.13 `HasBinaryProducts`  
+0.13 `Allegory.toCat`

$v_2$  —  $\sigma_2 = 81.8$  (5.3%)  
allegory vs plain category

+0.41 `Allegory.toCat`  
−0.32 `Cat`  
+0.24 `Alg.le`  
+0.23 `Allegory.recip`  
+0.22 `UnionAllegory...`  
+0.21 `distrib...isUnion`  
+0.19 `Cat.Hom`  
+0.18 `DistribAlleg...`

$v_3$  —  $\sigma_3 = 57.7$  (2.6%)  
limits vs subobjects

+0.33 `Cat`  
+0.30 `HasBinaryProducts`  
+0.29 `HasTerminal`  
+0.21 `Cat.Hom`  
−0.21 `Subobject.dom`  
−0.20 `Subobject.arr`  
−0.18 `Cat.assoc`  
−0.17 `Subobject`

All positive: everything uses the basic category operations. This block is the fact that the whole library rests on `Cat`.

A contrast: + on relational/allegory machinery, − on plain `Cat`. This axis separates Chapter-2 allegory code from category code.

A contrast: + on finite-limit structure (products, terminal), − on subobject machinery.

## The matching declarations $u_1, u_2, u_3$

$u_r$  lists the declarations that draw most on bundle  $v_r$  (entries are small — each is one of 9000).

$u_1$  — heaviest users of the core

- `amalgamation_is_pullback`
- `monic_epic_is_cover`

$u_2$  — the allegory side

- `Alg.powerRel_comp`
- `Alg.dp_thin_prefixed_context`

$u_3$  — the subobject side (−)

- `monic_epic_is_cover`
- `amalgamation_lemma`

- foldExists
- amalgamation\_lemma
- pushout\_monoid\_in\_pretopos
- capital\_iff\_complemented...
- recursor\_exists\_of\_bicartesian

Large high-level theorems that pull in a lot of category machinery.

- Alg.mapHasBinaryCoproducts
- Alg.A\_is\_map'
- Alg.elMap\_kappaMap\_disjoint
- Alg.dp\_thin\_prefixed
- Alg.A\_is\_map

Every one is Alg.\* — confirming  $v_2$  is the allegory-vs-category axis.

- amalgamation\_is\_pullback
- free\_peano\_property...
- foldExists
- peano\_property\_of\_bicartesian
- capital\_iff\_complemented...

Theorems that lean on subobjects (negative end of the limits/subobjects contrast).

## What $u_1$ is, as numbers

$u_1$  is a vector of 9135 real numbers — one per declaration — scaled so they square-sum to 1 (which is why each is small). All are  $\geq 0$  here (max = 0.0377); 2143 of the 9135 are essentially zero. The actual values, sorted:

| rank | $u_1$       | declaration                    |
|------|-------------|--------------------------------|
| 0    | +0.0377     | amalgamation_is_pullback       |
| 1    | +0.0355     | monic_epic_is_cover            |
| 2    | +0.0354     | foldExists                     |
| 50   | +0.0280     | Alg.mapDisjointBinaryCoproduct |
| 500  | +0.0195     | prodEndo_preservesProductMonic |
| 2000 | +0.0135     | PeanoProperty                  |
| 5000 | +0.0064     | Alg.globalSup_apply            |
| 9134 | $\approx 0$ | powerSetMap_id                 |

One entry means: **add up how core-ish each dependency that declaration  $i$  uses is, then divide by  $\sigma_1$**  —  $u_1[i] = \left( \sum_{j \text{ used by } i} v_1[j] \right) / \sigma_1$ . Worked for the top one: amalgamation\_is\_pullback uses 144 things; summing  $v_1$  over exactly those (0.508 + 0.414 + 0.394 + ...) gives 5.017, and 5.017/133.2 = 0.0377 — its  $u_1$  entry exactly. So  $u_1[i]$  is “how much of the category core declaration  $i$  pulls in.”

And the outer-product number: for  $i = \text{amalgamation\_is\_pullback}$  ( $u_1 = 0.0377$ ) and  $j = \text{Cat.Hom}$  ( $v_1 = 0.508$ ), block 1 gives  $133.2 \times 0.0377 \times 0.508 = 2.55$  (actual  $A_{ij} = 1$ ; block 1 overshoots, later blocks correct it). A light user (rank 5000,  $u_1 = 0.0064$ ) against Cat.Hom gives only 0.44 — small  $u_1$ , so the block predicts “barely uses the core.”

## What this buys us

The top block just rediscovers the core (the same hubs plain in-degree finds). The **later** blocks are more interesting: they split the library by subject — allegory vs category ( $v_2$ ), limits vs subobjects ( $v_3$ ) — which is a purely mechanical read of the dependency structure, no labels used.

The concrete payoff is for duplicate detection. Give each declaration its coordinates along  $u_1, \dots, u_{12}$  (its row of  $U$  scaled by the  $\sigma$ 's): a 12-number fingerprint of what it depends on. Two copies of the same lemma have the same checklist, hence the same fingerprint — the three known duplicates score cosine 0.94, 1.00, 1.00 in this 12-D space. So the SVD gives a fast way to **propose** duplicate pairs (nearest fingerprints); the final decision still compares the two rows directly (scripts/dep\_dup.py). The same coordinates would also give the graph viewer a deterministic layout.